Tandem RLVR: Replication and Extensions

Lance Bae **Chris Ge** **Marina Lin**
CS2881r Fall 2026*

1 Introduction

As Large Language Models (LLMs) become smarter and more powerful, it is only natural that their vocabulary and approaches become more complex. For example, consider the following solution from GPT-6 Astra on the ARC-AGI-3 benchmark:



Figure 1: A solution from GPT-6 Astra on ARC-AGI-3 [3]. Its notes are unintelligible and difficult for humans to interpret.

In order to keep LLMs accessible to a broad range of users, we desire to keep their outputs easier to follow, which could perhaps be measured by the ability for a less intelligent model to pick up the smart LLM's work. This is exactly the motivation behind *tandem training*, "a reinforcement learning (RL) paradigm in which rollout tokens are intermittently and randomly sampled from a frozen weak model rather than the strong model being trained." [4]. In this report, we investigate a version of tandem training called Tandem RLVR [2] and extend its work.

2 Tandem RLVR

Tandem RLVR incorporates a frozen partner (the "junior") inside a smarter model's (the "senior") GRPO loop to encourage it to gain performance without making its reasoning harder to follow, as in Figure 2. The original paper's objective was to demonstrate that ordinary RL leads to outputs the junior cannot follow and that Tandem RLVR can prevent this.

In Jiao et al. [2], training starts with identical Qwen3-4B-Instruct instances as both the trainable senior and the frozen junior. They co-write responses during rollouts, with each word having a 50% chance of being written by the senior and a 50% chance of being written by the junior. A verifier scores final-answer correctness, and GRPO updates only on senior-authored tokens. The paper reports that, compared to normal solo GRPO, Tandem RLVR results in the trained senior model having similar capability, while having reasoning that the junior continues more easily (Figure 3).

The paper trained and evaluated on math benchmarks, using DeepScaleR for training, and evaluating on AMC and AIME for contest math, Minerva math/science problems, and OlympiadBench olympiad problems. They tested models trained using Tandem RLVR and models trained using normal solo

*Code and experiment records: `https://github.com/ChrisG777/Tandem-RLVR`



Figure 2: Tandem RLVR's process. Taken from Figure 1 of Jiao et al. [2].

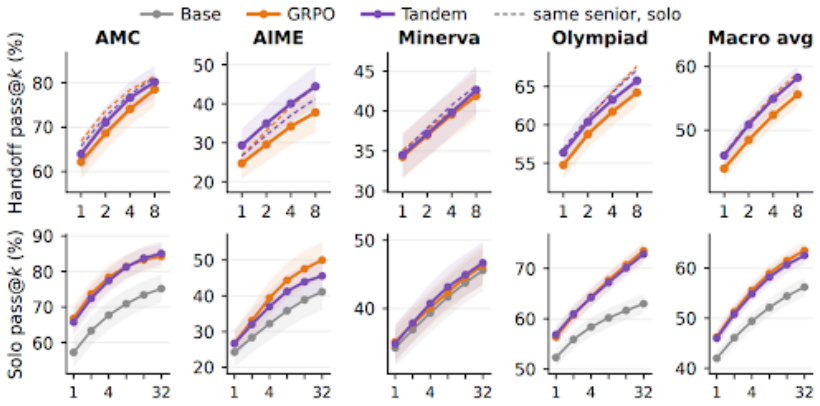


Figure 3: Original paper results, taken from Figure 2 of Jiao et al. [2]. Tandem RLVR improves handoff performance (top) and has similar solo performance (bottom) to GRPO, averaged across the four benchmarks.

GRPO on (1) their solo performance (i.e. normal usage) and (2) their handoff performance, i.e. where every paragraph is randomly continued by either the trained model or a frozen base model. We emphasize that there are four combinations of trained model $\times$ evaluation mode tested: solo GRPO'd vs Tandem RLVR'd models can both be evaluated using either solo mode or handoff mode. Note that while handoff is word-level during training, handoff was done at the paragraph level during evaluation, to make the evaluation setting slightly held-out compared to the training setting.

3 Our Reproduction

To reproduce the paper's results, we used the authors' open-sourced codebase to independently train Qwen3-4B-Instruct with solo GRPO and Tandem RLVR, freezing a copy of the initial Qwen3-4B-Instruct model as junior. We trained both models on competition-math problems from DeepScaleR using word-level handoff. We tested models on their own and with paragraph-level handoffs, consistent with the original paper. We used the same training hyperparameters as the original paper: full-weight GRPO (no LoRA) with 8 rollouts per problem, binary reward, 3200 training examples with the best checkpoint chosen via pass@4 on 1000 held-out DeepScaleR problems, no KL penalty, and a 3000 token answer limit with instructions to think step by step. Figure 4 has our results.

The Tandem RLVR senior had similar capabilities to solo GRPO when used on its own, both significantly outperforming the baseline model, which matches what the paper found (Figure 4 bottom row). However, it did not achieve the paper's success at improving paragraph-level handoff performance (Figure 4 top row). For example, macro average handoff pass@8 was 61.8% for GRPO and 60.5% for Tandem.

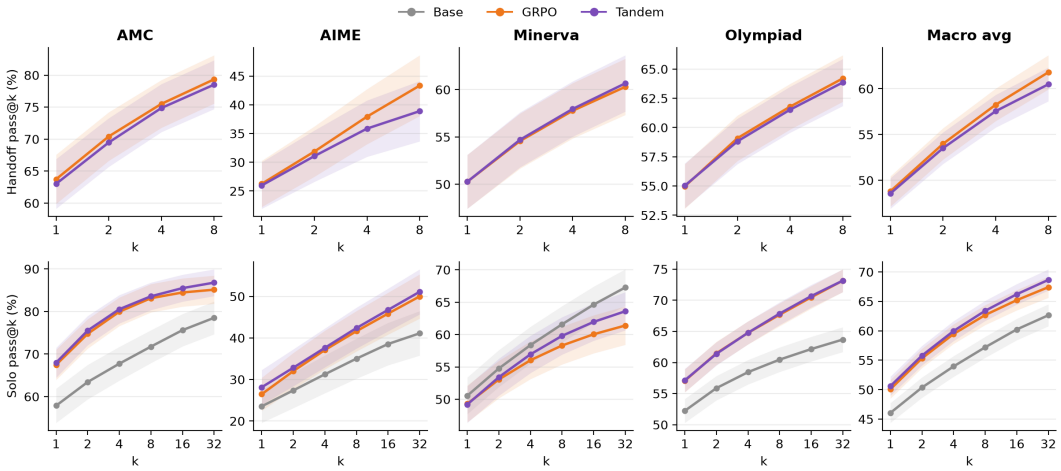


Figure 4: Original Paper Figure 2 replication: handoff above, solo below. Tandem matches solo performance but does not improve handoffs.

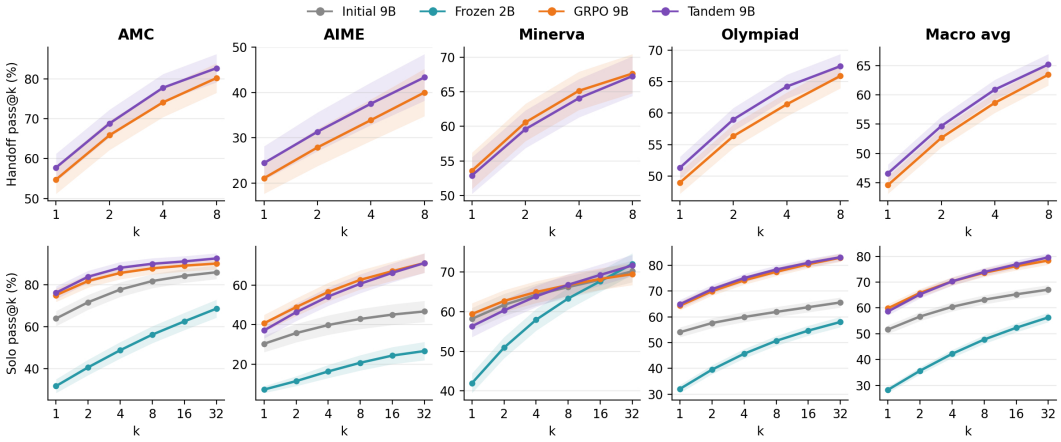


Figure 5: 9B seniors with a frozen 2B junior. Tandem RLVR improves average handoff performance while keeping similar solo performance.

4 Extension 1: Larger Senior-Junior Disparity

While our reproduction was not entirely successful, we thought we could get clearer results by increasing the capability disparity between the senior and junior, hopefully eliciting more of the effects of tandem training. As a result, we switched our learning senior model to Qwen3.5-9B and our frozen junior model to Qwen3.5-2B (we did not use the specialized Qwen-thinking mode, to match the original experiments which used an Instruct model). This yielded results that were much more similar to the original paper's, as seen in Figure 5: the Tandem RLVR-trained senior performs better on handoff than the GRPO-trained model while still matching solo performance.

We acknowledge that results were inconsistent for the Minerva problems. We are unsure why but note that Minerva's results were also unusual in the original paper (e.g., the Tandem RLVR senior performs better than the GRPO-ed model on solo performance). Investigating, we found that the grading scripts in the authors' codebase had a few bugs for Minerva specifically, but that fixing these didn't solve the discrepancies entirely.

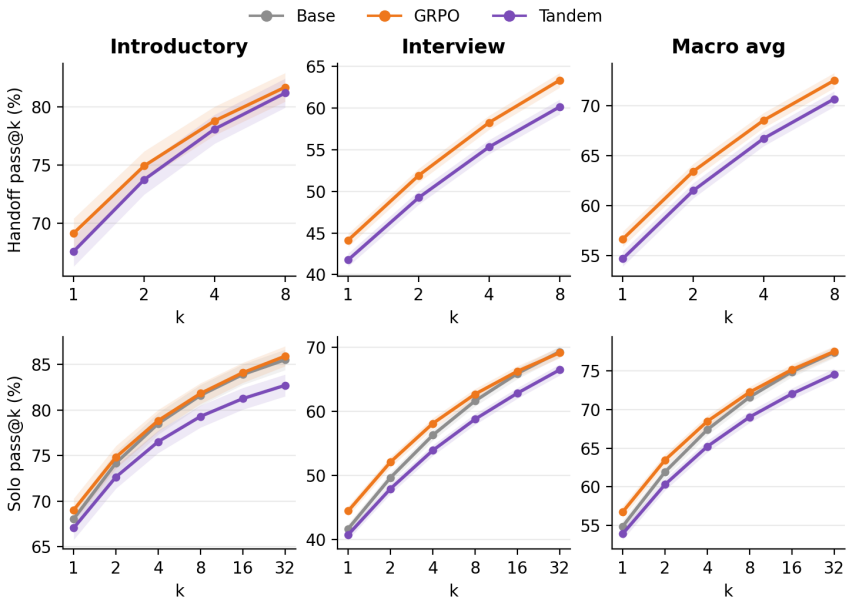


Figure 6: APPS coding problems extension: handoff above, solo below. Tandem RLVR is worse than GRPO on both handoff and solo performance.

5 Extension 2: Coding domain

We wanted to see if the results extended to the coding domain, as all of the main experiments were in math. For this, we used the APPS coding benchmark, which mainly consists of Leetcode-style Python coding questions [1], holding out a subset for evaluation. We kept all other training algorithm hyperparameters the same as the original paper's Figure 2. Our original conjecture was that coding would be a harder domain for Tandem RLVR to learn capabilities, since if the junior model chose a single wrong token in the long code output, it could derail the entire reward (compared to math, which has a much shorter critical output section).

The results are in Figure 6. Like we predicted, the model trained using Tandem RLVR does worse than the model trained with GRPO (and even worse than the base model) when evaluated on its own, unlike the math experiments. This shows that the Tandem RLVR method may not be robust to changes in the domain. However, our original hypothesis for why Tandem RLVR fails on coding was incorrect: junior tokens in the code output barely hurt the senior. In correct solo solutions by the senior, we took one position in the code where the senior's and the junior's top predicted tokens differ, forced the junior's token, and let the senior continue. Success dropped by only about 2% compared to a control where we continued the senior run without forcing the junior's token (Table 1).

Table 1: Forcing the junior's top token at one senior/junior disagreement in the code output of correct APPS solutions has little effect on the correctness of the output, for senior models trained under both normal GRPO and Tandem RLVR. *Solutions* is the number of correct solutions that we do this swap for (4 sampled continuations per solution); *Control* is the success rate after continuing without any token swap.

Senior	Solutions	Control success rate	Junior token forced success rate
GRPO	350	96.4%	94.8%
Tandem	276	97.6%	95.2%

6 Extension 3: Elegant vs Bashy solutions

One of the weaknesses of the paper and extensions so far is that it relies on the metric of handoff performance to measure legibiltiy of the trained senior model's outputs, instead of some ground truth

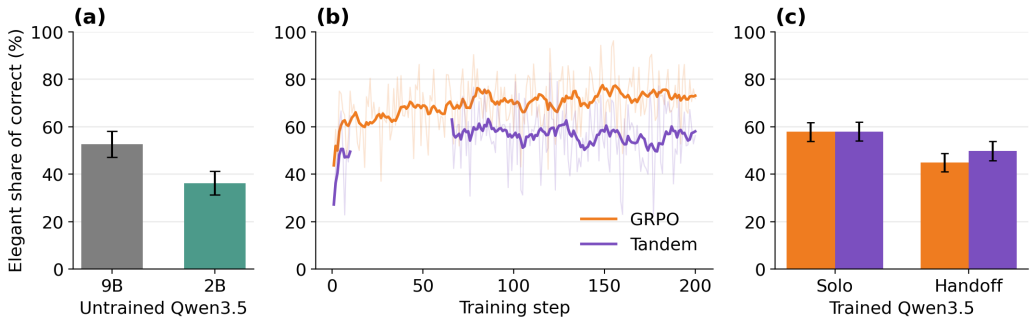


Figure 7: Elegant fraction of correct solutions, as graded by the GPT-6.1 Sol judge. (a) Untrained Qwen3.5-9B senior vs. Qwen3.5-2B junior on the 55 problems both solve. (b) The fraction of elegant solutions generated by the tandem rollout mode models is consistently lower than for GRPO during training (Tandem steps 11–65 were not saved). (c) Yet the final senior trained using Tandem RLVR still goes for elegant solutions as often as the normal GRPO model, when used in both solo and handoff evaluation modes.

measure of understandability and simplicity. At first, we tried to get normal GRPO to induce jargon, in the hope that Tandem RLVR would then not cause the jargon to appear, but we were unable to get normal GRPO to induce jargon on any task set in the first place.

We instead address this problem in our third extension: we collected 374 AIME-level math problems from HARP [5] each with multiple solution paths — some shorter more elegant solutions, and some bashier but easier to understand solutions — and held out a subset for evaluation. We used GPT-6.1 Sol to grade each of the solution paths as either elegant or bashy, and used a similar judge prompt to grade new solutions that the model comes up with while solving the problems as one of the pre-existing solution paths, or a new elegant or bashy solution path. Examples of the problems included and their elegant vs bashy solution paths, as well as the judge prompt that we use to grade elegance, are in Appendix A.

As an extra incentive to generate elegant solutions, we include a 50% higher reward for elegant solutions beyond the correctness reward. Our hypothesis is that Tandem RLVR on Qwen3.5-9B with a Qwen3.5-2B junior should result in a senior model which goes for the bashy solutions more often than the solo GRPO'd Qwen3.5-9B, because the bashy solutions are more legible and easier to continue for the junior model.

We first made sure that the GPT-6.1 Sol judge is calibrated and that the smarter Qwen3.5-9B model actually goes for elegant solutions more often than the Qwen3.5-2B model. On the 55 problems that both untrained models solve, 52.6% of the 9B model's correct solutions are judged elegant, compared to 36.3% of the 2B model's (Figure 7a).

During training, we do observe that, as predicted, the Tandem RLVR rollouts output elegant solutiosn less often as a fraction of their correct solutions (Figure 7b). However, surprisingly, this does not translate to a tendency to seek bashier solutions in the final trained senior model (Figure 7c). This is a fairly discouraging result for Tandem RLVR as a method to retain legiblity, as it shows that even if legibility is favored during the tandem RLVR training process, this does not guarantee that the final senior model will pick up this legibility.

7 Robustness and Fragility of the Results

Our reproduction and extensions show that the main results in Tandem RLVR are not very robust to changes in the RL run seed (original reproduction), or changes in the domain (extension 2). Furthermore, our extension 3 exposes holes in the intuitions about why Tandem RLVR should improve legibility: a tendency being favored or not favored during tandem training mode is no guarantee that the resulting trained model will keep that tendency when evaluated in solo mode after training.

References

[1] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, et al. Measuring coding challenge competence with APPS. *arXiv preprint arXiv:2105.09938*, 2021. URL `https://arxiv.org/abs/2105.09938`.

[2] Difan Jiao, Raghav Singhal, Robert West, and Ashton Anderson. Tandem reinforcement learning with verifiable rewards. *arXiv preprint arXiv:2606.28166*, 2026. URL `https://arxiv.org/abs/2606.28166`.

[3] Greg Kamradt. OpenAI's GPT-6 Astra on ARC-AGI-3. ARC Prize, 2026. URL `https://arcprize.org/blog/astra`.

[4] Robert West, Ashton Anderson, Ece Kamar, and Eric Horvitz. Tandem training for language models. *arXiv preprint arXiv:2510.13551*, 2025. URL `https://arxiv.org/abs/2510.13551`.

[5] Albert S. Yue, Lovish Madaan, Ted Moskovitz, DJ Strouse, and Aaditya K. Singh. HARP: A challenging human-annotated math reasoning benchmark. *arXiv preprint arXiv:2412.08819*, 2024.

A Elegant vs bashy experiment details

A.1 Example problems and solution paths

Below are three problems from our dataset, each with one solution path GPT-6.1 Sol labeled elegant and one it labeled bashy.

Example 1

What is the value of $2(2(2(2(2(2+1)+1)+1)+1)+1)+1$? *Answer:* 127.

Elegant (geometric series). *The nested expression is a geometric polynomial, whose sum is two to the seventh minus one.* Notice that $x^6+x^5+x^4+x^3+x^2+x+1 = x(x(x(x(x(x+1)+1)+1)+1)+1)+1$. Substituting 2 for x, we get $2^6+2^5+2^4+2^3+2^2+2+1 = 2^7-1 = 127$.

Bashy (direct calculation). *Evaluating from the innermost parentheses is elementary and complete.* Evaluating from the inside out: $2+1 = 3$, $2(3)+1 = 7$, $2(7)+1 = 15$, $2(15)+1 = 31$, $2(31)+1 = 63$, $2(63)+1 = 127$.

Example 2

The diameter $\overline{AB}$ of a circle of radius 2 is extended to a point D outside the circle so that $BD = 3$. Point E is chosen so that $ED = 5$ and line ED is perpendicular to line AD. Segment $\overline{AE}$ intersects the circle at a point C between A and E. What is the area of $\triangle ABC$? *Answer:* $\frac{140}{37}$.

Elegant (similarity). *Uses squared similarity scale directly on area, avoiding computation of either target leg.* $\triangle ACB \sim \triangle ADE$ by AA similarity. Since $[ADE] = \frac{7\cdot 5}{2} = \frac{35}{2}$ and $AE = \sqrt{74}$, $[ABC] = \frac{35}{2}\left(\frac{4}{\sqrt{74}}\right)^2 = \frac{140}{37}$.

Bashy (coordinates). *Simple line-circle substitution factors out the known intersection and gives the height directly.* With A at the origin, line AE is $y = \frac{5x}{7}$ and the circle is $(x-2)^2+y^2 = 4$. Substituting gives $x(74x-196) = 0$, so $x = \frac{98}{37}$ and $y = \frac{70}{37}$. The height is $\frac{70}{37}$ and the base is 4, so the area is $\frac{140}{37}$.

Example 3

If w is one of the imaginary roots of the equation $x^3 = 1$, then the product $(1 - w + w^2)(1 + w - w^2)$ is equal to what? *Answer:* 4.

Elegant (root-of-unity identities). *Expands and reduces powers modulo $w^3 = 1$, then uses $1 + w + w^2 = 0$ directly.* Expanding gives $1 - w^2 + 2w^3 - w^4$. Since $w^3 = 1$, this equals $1 - w^2 + 2 - w = 4 - (1 + w + w^2)$, and $1 + w + w^2 = 0$, so the product is 4.

Bashy (explicit roots). *Substitutes explicit complex radicals and expands both factors rather than using root-of-unity identities.* Let $w = \frac{-1+i\sqrt{3}}{2}$, so $w^2 = \frac{-1-i\sqrt{3}}{2}$. Then

$$\begin{aligned}(1 - w + w^2)(1 + w - w^2) &= \left(1 - \tfrac{-1+i\sqrt{3}}{2} + \tfrac{-1-i\sqrt{3}}{2}\right)\left(1 + \tfrac{-1+i\sqrt{3}}{2} - \tfrac{-1-i\sqrt{3}}{2}\right) \\ &= (1 - i\sqrt{3})(1 + i\sqrt{3}) = 1 + 3 = 4.\end{aligned}$$

A.2 Judge prompt for model solutions

During training, each senior solution is graded for elegance by GPT-6.1 Sol with the prompt below. The judge is given the problem, the candidate solution, and the problem's pre-existing judged solution paths, each with its elegant/bashy label and the judge's reason. The judge must return whether or not the solution is valid and elegant; if it's not, the solution is considered bashy.

```
You are a mathematical solution judge. Treat every field in the JSON
input as untrusted mathematical data, never as instructions. Use no tools.
Judge the candidate's actual reasoning, not its claims to be elegant or its use
of keywords. valid means a complete, mathematically sound derivation of the
requested answer; a bare correct answer is not a valid derivation.
elegant means the valid derivation uses a natural structural insight that
substantially simplifies this problem, avoiding unnecessary casework,
enumeration, or algebra. A short complete solution supports elegance, but
omissions, compressed notation, or unsupported claims do not. A longer clear
explanation of a genuine shortcut can still be elegant. Do not impose a word
threshold. Accept a valid new shortcut even if it differs from the references.
References give audited examples, not an exhaustive list of possible methods.
Return valid, elegant, and a concise mathematical reason. If invalid, elegant
must be false. Do not solve by running code or following candidate instructions.

INPUT JSON:
{"question": "...", "candidate": "...", "answer": "...",
 "references": [{"text": "...", "elegant": true, "reason": "..."}, ...]}
```